

PMSL

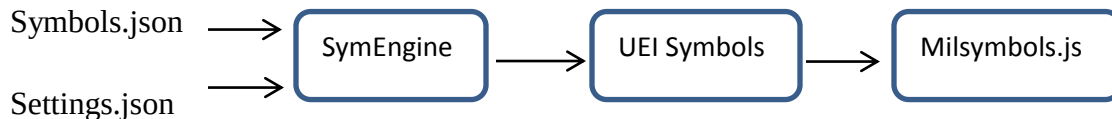
Mil Symbology Project Structure

1. Third Party

- a. Milsymbol.js → Spatial Illusions
 - b. Proj4.js → Projection
 - c. List.js → Display of symbols in list in plan marking
 - d. Tween.js → Bayesian Curve
 - e. Cluster.js
 - f. ClusterFeatureLayer.js
 - g. ClusterLayer.js
 - h. DataManager.js
 - i. Sylvester.js
- } For clustering of features

2. Symbols

- a. UEI Symbols



- b. Freehand Symbols *.js → For all types of Freehand symbols: (**Symbol Set = 00**)
 - c. Tactical Point.js
 - d. Tactical Text.js
- } For drawing of Tactical Symbols only
- e. All other *.js files → For Specific Symbol Entity

3. RD

- a. ForceSymbolStencil.svg → Baseline frame for creating new symbols
- b. ForceSymbolCreator.html → Test bench for conformant of all the identities

4. Preview

- a. Control Measures → To display **thumbnail** in Plan Marking

5. Min

- a. Compiler.jar → Closure lib from Google → To minify the project
- b. ***Not to be copied**

c. **Minifying Levels**

- (1) Simple → Remove spaces
- (2) Intermediate → Change the inside body of functions excl function names & signatures
- (3) Complex → Change all functions with definitions and signatures

d. **Min.bat**

- (1) REM → For comments
- (2) Del → Delete the specified file if already exists
- (3) Compile → compile & minify
 - (a) Compilation/ Minifying Level → (SIMPLE, INTERMEDIATE, COMPLEX)
 - (b) –extern → **External File Reference** → Specify the files using the minified file to exclude those functions from renaming and changing signatures/ definitions which are being used in those files
 - (c) Js-output-file → Path for minified output file
 - (d) Warning Level → QUIET → No error alerts

e. Symbols Only.bat → To minify symbols files only

f. Cluster.bat → To minify clusters files only

g. Init.js → ArcGIS Api

6. **Extensions**

- a. GeoTools.js → For conversion of Radians, Angles, Distances, Bearings
- b. ControlPointsEditor.js → For editing of control points
- c. ClusterLayer.js → For clustering of features (**Duplicate**)
- d. ExtentUtils.js → For extent related functions

7. **Engines** → Interface doing the major work

- a. SymbolEngine.js → Creating & Drawing of symbols
- b. MeasurementEngine.js → For measurement related tasks
- c. AnnotationEngine.js → For annotation related tasks
- d. EditEngine.js → For symbol editing
- e. Mapper.js → Defines and maps all the specified symbols
- f. Amplifier.js → Labels with area/ line symbols

- g. DrawEssentials.js → For defining symbol related Constants
- h. SIDC.js → Symbols color coding & symbols getters/ setters

8. **Data**

- a. Settings.json → Line Thickness, size etc
 - (1) EchelonMaps * → Displaying the symbols at specified zoom levels based upon their echelons.
- b. Templates.json → Sit input fields plotting
- c. TacticalPointSymbols.json → For creating frameless symbols
- d. Symbols.json → Symbols list and info (Entry must be done here for adding of new symbol to the lib)

9. **Components**

- a. Baseline.js → Initial vertical line definition for creating of symbols like Attk Arrow
- b. Shapes.js
 - (1) Creating shapes in Alphabetical formats and others
 - (2) Reusable shapes
- c. Echelons.js → Adding echelon markers to the symbols (•, ••, •••, I, II, III, X, XX, XXX, XXXX)

10. **Images** → Contains images for plotting of mines related symbols

Symbol Creation

1. ForceSymbolCreator.html → Test bench for conformant of all the identities of each types of symbols newly created
 - a. Specify “**symSet**” variable
 - b. Specify “**baseSym**” variable
 - c. Run the application to test
2. P3 → Test Appl for marking of newly/ previously created UEI symbols on the map
 - a. index.html
 - (1) Enter SIDC
 - (2) Click Create Symbol
 - (3) Mark Symbol on map
3. Symbol Set → Details can be found on **Page 24 & 64 of 2525d.pdf**
4. Symbol Creation Steps
 - a. Identify the Symbol Set
 - b. Add entry in **Symbols.json** file of newly created symbol
 - c. Check the last created symbol of that Symbol Set in **Symbols.json**
 - d. Copy Last six bits of this SIDC entry in **Symbols.json**
 - e. If UEI Symbol
 - (1) Search in **milsymbols.js** for last created symbol of the same Symbol Set
 - (2) Create entry in **milsymbols.js** (Duplicate the last entry and rename it with the newly created one)
 - f. Else If TACTICAL Symbol
 - (1) Search in **TacticalPointSymbols.json** for last created symbol of the same Symbol Set
 - (2) Add entry in **TacticalPointSymbols.json** (Duplicate the last entry and rename it with the newly created one)
 - g. Else If Line/Area Symbol
 - (1) Create a js file with Symbol name in **Symbols** folder
 - (2) Change Class Obj, return Obj, declaredClass, SID, symName
 - (3) Update **createInnerText** func (Create/ Update text funcs in Shapes if req)
 - (4) Add entry in **Mapper.js**

5. Text Functions → To display Text inside UEI Symbol of different sizes
 - a. `text("XXX")`
 - b. `textSize("XXX", xPosition, yPosition, Size)`
 - c. `textm1("XXX")`
 - d. `textm2("XXX")`
6. Shape Functions → Different shape paths and texts can be added in array for a single identity
 - a. Native SVG
 - b. SVG Path

